Test assignment: Text-to-Wikidata Entity Linking Pipeline

1. Objective

Build a simple Python pipeline that identifies meaningful terms in a text, retrieves candidate concepts from Wikidata, and uses a GraphRAG-inspired approach based on RDF (SKOS) to select the concept that best matches each term in its original context.

Goal: the pipeline should transform unstructured text into structured entity links, with each linked term pointing to a Wikidata entity (QID), such as Q42 for Douglas Adams.

Core principle: candidate retrieval finds plausible concepts; graph-based contextual reasoning helps determine which concept is the right one.

2. Scope and expected outcome

Input

A plain-text document, for example:

> Marie Curie conducted research on radioactivity in Paris and won the Nobel Prize in Physics.

Output

A structured list of terms linked to Wikidata concepts.

Mention	Candidate concept	Wikidata ID	Confidence
Marie Curie	Marie Curie	Q7186	0.99
radioactivity	Radioactivity	Q11452	0.96

Paris	Paris	Q90	0.98
Nobel Prize in Physics	Nobel Prize in Physics	Q38104	0.98

The IDs and scores above are illustrative expected output; scores must be generated by the implemented system.

3. Explain the architecture and your implementation (in 1-2 pages)

The architecture document should consist of the following components:

1. **Text processing and term extraction:** Process the input text and extract relevant terms, named entities, and noun phrases. Preserve the original text and the position of each extracted term.
2. **Candidate retrieval:** Use the Wikidata API to retrieve a set of candidate concepts for each term. Candidates should include their Wikidata IDs (QIDs), labels, descriptions, and aliases. The retrieval stage should prioritize finding the correct concept among the candidates rather than making the final selection.
3. **Knowledge graph enrichment:** Retrieve relevant properties and relationships for each candidate from Wikidata. Build a small graph connecting candidate entities through their known relationships. This graph should provide contextual information for distinguishing between concepts with similar labels.
4. **GraphRAG-based candidate selection:** Use semantic similarity between the input context and candidate descriptions, together with graph relationships, to rank the candidates. The implementation should compare candidates based on textual relevance, entity types, and connections to other concepts mentioned in the text. The highest-ranked candidate should be selected when sufficient evidence is available.
5. **Output and evaluation:** Return the original terms, selected Wikidata concepts, QIDs, ranking scores, alternative candidates, and supporting evidence in a structured JSON format. Evaluate the pipeline on a small test dataset containing both ambiguous and unambiguous terms.

The implementation may use Python, spaCy for term extraction, the Wikidata API for candidate retrieval, sentence embeddings for semantic similarity, and NetworkX for representing the local knowledge graph. The solution should remain lightweight and modular, without requiring a complete local copy of Wikidata.

The evaluation should measure candidate retrieval quality and final entity-linking accuracy. In particular, compare the results of a baseline ranking method without graph information against

the GraphRAG-based approach to determine whether graph context improves the selection of the correct Wikidata concept.

The final deliverable should include the implemented pipeline, a description of the architecture and ranking strategy, example input and output, and an evaluation of the results.

In scope

- Extract entity mentions and meaningful terms from text.
- Retrieve multiple Wikidata candidates for each mention.
- Enrich candidates with labels, descriptions, aliases, and graph relationships.
- Rank candidates using textual relevance and graph context.
- Return the best match, alternatives, confidence, and an explanation.
- Evaluate the pipeline on a small test dataset.

Out of scope for the MVP

- Building a complete Wikidata knowledge graph locally.
- Indexing all Wikidata entities and relationships.
- Training a custom language model.
- Fully automated linking of every possible noun phrase.
- Production-scale distributed processing or real-time graph updates.